



PAW

Lic. en Sistemas de Información

Trabajo Práctico Final

Integrador de Conocimientos

Grupo Cerberus

Integrantes:

Salvador Baez

Mateo Nomico

Tomás Resnik



PAW

Lic. en Sistemas de Información

Arquitectura y Modelo de Objetos IndieCinema

Trabajo Práctico Integrador · Entrega 2 · Grupo Cerberus



indie
CINEMA



1. Alcance de este documento

Se realizaron los wireframes del sitio, la definición de la arquitectura de la aplicación y el diseño del modelo de objetos del back-end. Los wireframes van en un archivo aparte (`/wireframe/home.png...`), este documento cubre las otras dos

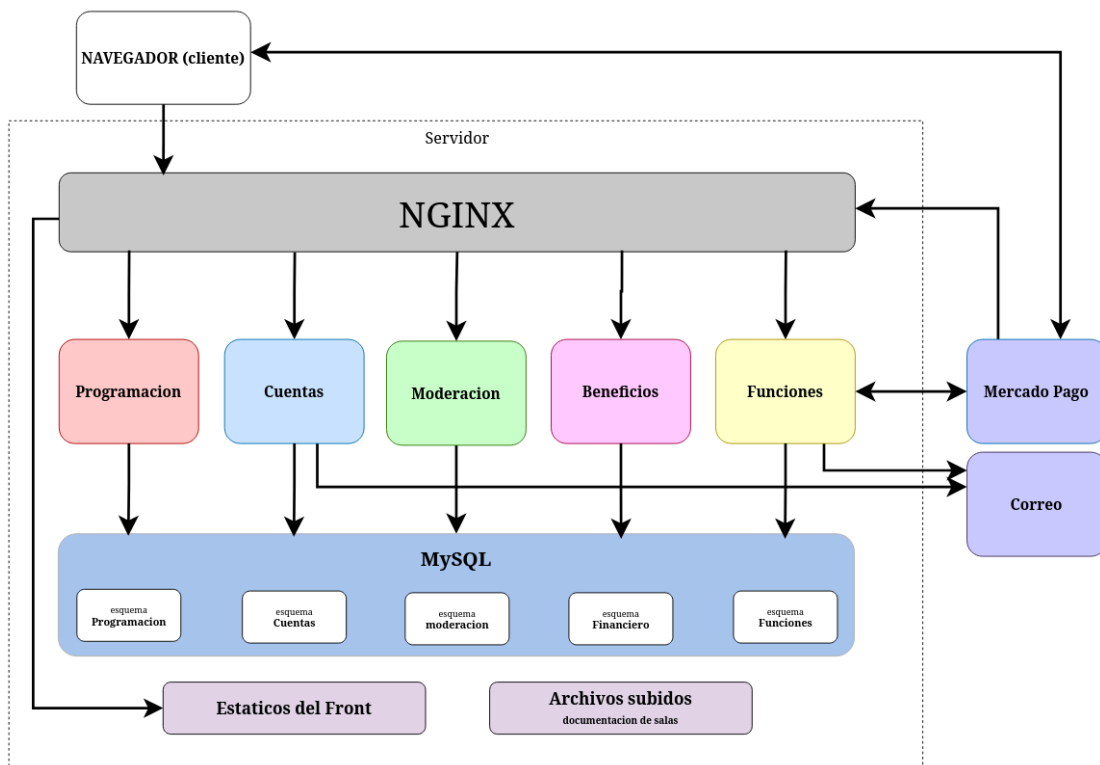
Se parte con cinco subsistemas, un usuario de base por subsistema, y se lo baja un nivel: cómo interactúan cliente, servidor y servicios externos; qué expone y qué consume cada subsistema; cómo es un subsistema por dentro; y qué clases tiene cada uno. Dos cosas se fijan acá: el código se reparte en dos repositorios, uno de front-end y uno de back-end, y cada subsistema es una aplicación PHP con patrón MVC, construida con un núcleo propio y Twig o con Laravel (a decidir). la arquitectura es la misma en los dos casos.

2. Vista general: cliente, servidor y servicios externos

IndieCinema es una aplicación web de tres partes. El cliente es el navegador: recibe HTML ya armado, con el CSS de la identidad visual y el JavaScript propio que hace interactivos el carrusel de la cartelera, los filtros y el arrastrar y soltar del panel del organizador. El front-end no corre por separado sino que es un paquete de plantillas, CSS y JS que cada subsistema instala y con el que arma sus páginas en el servidor. El JS mejora lo que ya anda (valida formularios, pide JSON al mismo subsistema para refrescar los lugares libres sin recargar).

El servidor es un único host con un único dominio. Adelante hay un nginx que termina HTTPS, valida la sesión con cuentas y enruta cada petición, según el prefijo de la URL, a uno de los cinco subsistemas PHP. Cada subsistema es una aplicación MVC independiente en su propio contenedor. Los cinco comparten una instancia de MySQL, pero cada uno tiene su esquema y su usuario de base con permisos sólo sobre lo suyo. Los estáticos del paquete front los sirve nginx directamente; los archivos que suben los usuarios (la documentación de las salas) viven en un volumen fuera del webroot y los entrega moderación tras verificar el rol.

Los servicios externos son dos: Mercado Pago, que cobra las entradas (el espectador paga en el checkout de Mercado Pago y la plataforma se entera por un webhook que entra por nginx a funciones), y un servidor de correo al que escriben cuentas (confirmación de registro) y funciones (reserva confirmada, recordatorio de función).



draw.io (hoja 1) :

<https://drive.google.com/file/d/1AQqtOWbPA1VYGsZCJ0kpSpm-PVMIR3i8/view?usp=sharing>

Componente	Tecnología
Navegador	HTML5, CSS3 y JavaScript sin framework
nginx	Reverse proxy: HTTPS, ruteo por prefijo, validación de sesión, límite de intentos, estáticos
Subsistemas	PHP 8 con MVC: núcleo propio (PSR-1/12, PSR-4, PSR-3) + Twig, o Laravel. un contenedor por subsistema
Paquete front	Paquete Composer <code>indiecinema-front</code> : plantillas, CSS de la identidad y componentes JS
Base de datos	MySQL 8: un esquema y un usuario por subsistema; consultas preparadas; vistas de sólo lectura como contrato entre esquemas
Mercado Pago	Checkout Pro + webhook; credenciales sólo en funciones
Correo	SMTP externo
Entorno	Docker Compose: nginx, cinco contenedores PHP, MySQL y volúmenes

3. Despliegue, sesión y repositorios

Un `docker compose up` levanta todo. En desarrollo corre en la máquina de cada integrante con datos de prueba; en producción, igual, en un VPS con el dominio real y un certificado de Let's Encrypt. La única diferencia es el archivo de configuración de cada subsistema (`ini` con núcleo propio, `.env` con Laravel). ahí van la base, el prefijo, las URL internas de los otros subsistemas y los secretos.



Prefijo	Subsistema	Esquema y usuario de base	Entrega (prevista)
/	programación: catálogo, salas, funciones, cartelera, votación de títulos, rankings	programacion	3
/cuenta/	cuentas: registro, sesión, roles, perfil, seguidos	cuentas	3
/funcion/	funciones: acuerdo de fecha, reserva y cupo, cobro, asistencia, calificaciones	funciones	3
/moderacion/	moderación: admisión de salas, revisión, clasificación, denuncias, términos	moderacion	4
/Beneficios/	Beneficios: reparto, liquidaciones, puntos, comercios asociados, reportes	Beneficios	4
/estaticos/	CSS, JS e imágenes del paquete front (los sirve nginx)	—	3

Programación queda en la raíz porque es el contenido público y el que más se comparte y se indexa. Cada subsistema recibe la ruta sin su prefijo y lo repone al generar links, leyéndolo de su configuración.

Dos repositorios: front-end y back-end

`indiecinema-front` es un paquete Composer, por lo que no corre solo, lo instala cada subsistema. Contiene la plantilla base (encabezado, menú, pie), los parciales comunes, una carpeta de plantillas por subsistema, el CSS de la identidad y los componentes JavaScript, sin paso de compilación.

`indiecinema-back` es un monorepo con el `docker-compose.yml`, la configuración de nginx, el script inicial de MySQL (esquemas, usuarios, permisos y vistas) y una carpeta por subsistema. Cada subsistema declara el paquete front en su `composer.json`.

4. El back-end por dentro: cinco subsistemas

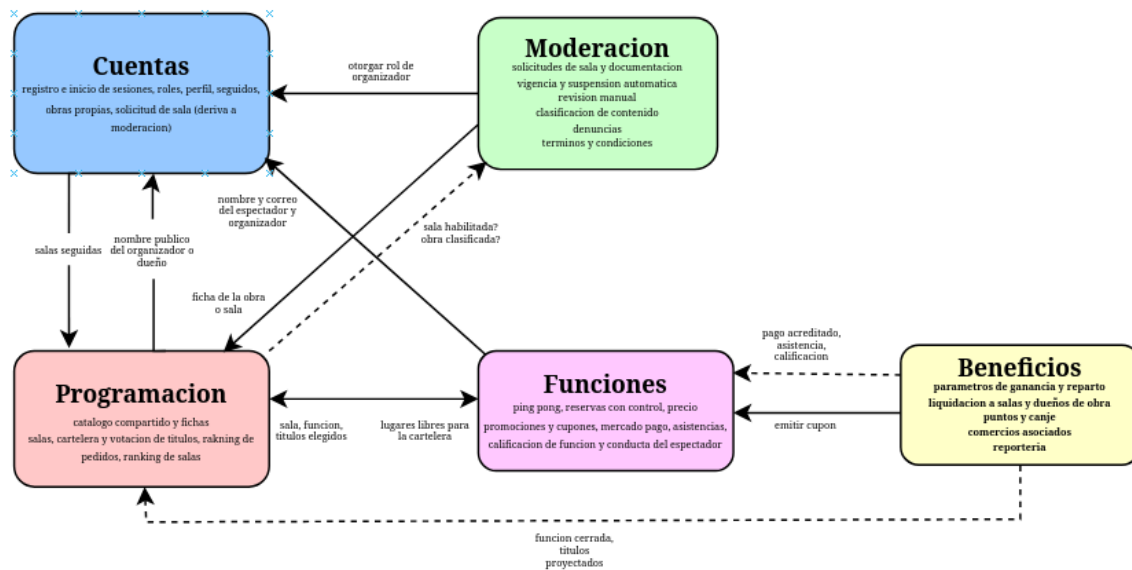
El criterio de corte es por función de sistema, y cada corte se justifica por mínimo privilegio (ningún subsistema ve datos que no necesita).

(hoja 2 del draw.io)



PAW

Lic. en Sistemas de Información



Subsistema	Es dueño de	Expone	Consume
cuentas	Usuario, PerfilUsuario, Rol (y la tabla técnica de sesiones)	GET /interno/sesion → usuario y rol (lo llama nginx) GET /interno/usuarios/{id} → nombre público y correo POST /interno/usuarios/{id}/roles (sólo moderación)	programación: nombre de las salas seguidas, para «Mi cuenta». Es la única llamada que sale de cuentas.
moderación	SolicitudSala (con su documentación), Denuncia; términos y condiciones y clasificación de contenido	vistas v_salas_habilitadas y v_obras_clasificadas, que programación lee antes de dejar programar o proyectar	cuentas: otorgar el rol de organizador al aprobar programación: ficha de la obra o la sala que revisa o clasifica
programación	Sala, CategoriaSala, ObraAudiovisual (Película, Cortometraje), GeneroObra, PedidoTitulo, FuncionCartelera	GET /interno/funciones/{id} → sala, estado, títulos, precio base, cupo, organizador y dueño POST /interno/funciones/{id}/programar → fija la fecha acordada GET /interno/salas, GET /interno/obras/{id} → nombres y fichas vista v_funciones_cerradas (con títulos proyectados)	cuentas: nombre público del organizador o del dueño moderación: ¿sala habilitada?, ¿obra clasificada? funciones: lugares libres para la cartelera; crear el acuerdo de fecha con las franjas propuestas
funciones	AcuerdoFechaPingPong, Reserva (con su pago), Calificación;	GET /interno/funciones/{id}/ocupacion → lugares libres y calificación promedio	cuentas: nombre y correo del espectador y del organizador



	promociones y cupones de descuento	POST /interno/cupones → emite un cupón de descuento (lo llama Beneficios) vistas v_pagos_acreditados, v_asistencias, v_calificaciones	programación: datos de la función; fijar la fecha acordada Mercado Pago, correo
Beneficios	RepartoGanancia, BilleteraPuntos, BeneficioComercio, Cupon (vale de comercio); parámetros de reparto y reportes	sólo pantallas: nada por API	funciones: emitir cupón de descuento; lectura de sus vistas (pago acreditado, asistencia, calificación) programación: lectura de v_funciones_cerradas (función cerrada, títulos proyectados)

Dos formas de hablar entre subsistemas

- **Llamadas a la API interna (flechas llenas):** HTTP/JSON sobre la red interna del compose, en rutas bajo /interno/ que nginx no publica. Cada llamada lleva el token del subsistema que llama y cada ruta interna declara qué llamadores acepta, el receptor rechaza el resto. Son consultas de lectura o comandos puntuales.
- **Vistas de sólo lectura (flechas punteadas):** para el historial, no hay llamadas: el dueño de los datos publica una vista SQL sobre sus tablas y el usuario de base del otro recibe `GRANT SELECT` sólo sobre esa vista, nunca sobre las tablas. La vista es el contrato: si funciones cambia sus tablas, mantiene la vista. Beneficios corre un cierre periódico que lee las vistas y arma las liquidaciones y los puntos en su propio esquema; programación lee las de moderación para saber si una sala está habilitada o una obra clasificada. Consideramos una cola de mensajes y la descartamos por ahora por que es infraestructura extra y las vistas dan el mismo aislamiento con el privilegio declarado en SQL.

5. Modelo de objetos

El modelo tiene una clase por concepto del dominio y cada clase pertenece a un solo subsistema: es el mismo corte del mapa de la sección 4 llevado a los objetos, y es lo que después baja a las tablas del esquema de cada uno. Criterios que se aplicaron en todo el diagrama:

- **Un dueño por clase.** Ningún objeto se comparte entre subsistemas. Cuando uno necesita referirse a algo de otro guarda su identificador (`organizadorId`, `funcionId`, `duenoId`) y, si necesita más, pregunta por la API interna o lee la vista.



PAW

Lic. en Sistemas de Información

- **Identificadores UUID** generados por la aplicación: cruzan subsistemas sin colisiones, no revelan cuántas reservas o usuarios hay y sirven como referencia externa hacia Mercado Pago.
- **Estados como enumeraciones** (`EstadoFuncion`, `EstadoReserva`, `EstadoSolicitud`, `EstadoDenuncia`, `EstadoPedido`): las transiciones válidas las hace cumplir el servicio de dominio, y los ciclos de vida están más abajo, en «Ciclos de vida».
- **Herencia sólo donde hay un tipo real:** `ObraAudiovisual` es abstracta y se concreta en `Pelicula` y `Cortometraje`; el resto son asociaciones y composiciones.
- **Los métodos son las operaciones del dominio**, no getters: `Reserva.confirmarPago()`, `SolicitudSala.aprobar()`, `BilleteraPuntos.canjearBeneficio()`. Cada uno corresponde a una acción de los wireframes o a un evento de otro subsistema.

Diagrama de clases:

<https://mermaid.ai/d/90896f9b-309b-4746-84d3-f3b2270a9706>

	nginx descarta las cabeceras X-Usuario-* entrantes y limita los intentos por IP en inicio de sesión, registro, votos y webhook de pago.	Un pedido con X-Rol: administrador puesto a mano ve lo mismo que un visitante.
Sistema operativo	VPS con una distribución de soporte largo y actualizaciones de seguridad automáticas; SSH sólo con clave, sin root y con fail2ban. Imágenes oficiales de nginx, php:8-apache y mysql:8, reconstruidas en cada entrega; ningún contenedor privilegiado ni con el socket de Docker montado. El volumen con la documentación de las salas se monta sólo en moderación. Cada subsistema lee sus secretos de su propio archivo de configuración, que no se versiona.	ssh root@... es rechazado, y también el ingreso con contraseña. docker compose config muestra el volumen de documentación montado únicamente en moderación. docker compose exec programacion env grep MP_ no devuelve nada; en funciones, sí. git grep sobre los dos repositorios no encuentra contraseñas ni tokens.
Base de datos	Un usuario por subsistema con permisos sólo sobre su esquema, más GRANT SELECT sobre las vistas v_* que consume. Las vistas se crean con SQL SECURITY DEFINER, así que el lector no necesita permisos sobre las tablas de origen. El usuario con el que corre la aplicación no tiene CREATE, ALTER ni DROP; las migraciones usan otro. Claves únicas que sostienen reglas aunque falle el código: id de pago de Mercado Pago, un voto por usuario y función, un compromiso por usuario y acuerdo. Respaldo diario con mysqldump, cifrado y guardado fuera del VPS.	Conectado como programacion, SELECT * FROM cuentas.usuario devuelve ERROR 1142. SHOW GRANTS de cada usuario coincide con la matriz de permisos. Insertar dos veces el mismo id de pago falla por la clave única. Antes de cada entrega se restaura el último respaldo en una base limpia.
Aplicación	cuentas: Argon2id; cookie HttpOnly, Secure y SameSite=Lax; id de sesión regenerado al entrar; recuperación con token de un solo uso que vence. Middleware de cada subsistema: rol mínimo por ruta y token CSRF. El servicio verifica además que el recurso sea del usuario. Consultas preparadas y autoescape de Twig; logs por subsistema sin datos personales y auditoría de cambios de rol, decisiones de moderación y liquidaciones. funciones: el webhook valida la firma x-signature, consulta el pago a la API de Mercado Pago y compara el monto con la reserva. moderación: los PDF se validan por contenido y tamaño, se guardan con un UUID fuera del webroot y los entrega un controlador que verifica el rol.	Pruebas automáticas por rol: un espectador contra /organizador/... recibe 403, y un organizador contra la sala de otro, también. Webhook sin firma válida: 401. El mismo aviso dos veces: una sola confirmación. Un formulario enviado sin token CSRF es rechazado. Un .php renombrado a .pdf no se acepta como documentación. Análisis de OWASP ZAP (zap-baseline.py) sobre el sitio desplegado, sin alertas de riesgo alto.

Seguridad en cada etapa del ciclo de vida

Diseño	Corte en subsistemas por mínimo privilegio y radio de daño; cada control asignado a un componente; matriz de permisos de la base; amenazas del circuito de pago resueltas en la secuencia de la sección 7 (firma, consulta a la API, idempotencia, bloqueo del cupo); revisión del diseño con los tres ajustes anteriores.
--------	--